تحليل وتصميم الخوارزميات

المحاضرة الثانية: تحليل تعقيد الخوارزميات

سنتعمق في أحد أهم جوانب تصميم الخوارزميات وهو "التحليل".

سنطرح سؤالاً جوهرياً: عندما يكون لدينا أكثر من خوارزمية لحل نفس المشكلة، كيف نحدد أيها "الأفضل"؟ الجواب يكمن في تحليل كفاءتها، وهو ما يُعرف بـ **تحليل التعقيد.(Complexity Analysis)**

لماذا نحلل الخوارزميات؟

تخيل أنك تبني تطبيقاً يستخدمه آلاف الأشخاص. إذا كانت الخوارزمية التي يعتمد عليها التطبيق بطيئة، فستكون تجربة المستخدم سيئة جداً. تحليل التعقيد يساعدنا على:

- **المقارنة بين الخوارزميات :**يمنحنا طريقة موضوعية لاختيار الحل الأكثر كفاءة.
- **توقع الأداء :**يمكننا من تقدير كيف سيتصرف البرنامج مع زيادة حجم المدخلات (مثلاً، زيادة عدد المستخدمين أو البيانات).
- **تجنب المشاكل المستقبلية :**يساعدنا على كتابة كود قادر على التوسع والعمل بكفاءة مع نمو التطبيق.

مقاييس الكفاءة الأساسية

عندما نحلل كفاءة خوارزمية، فإننا نركز على موردين أساسيين يستهلكهما الحاسوب:

1. **تعقيد الوقت :(Time Complexity)** وهو مقياس لمقدار الوقت الذي تستغرقه الخوارزمية لتكتمل كدالة في حجم المدخلات. لا نقيس الوقت

بالثواني أو الدقائق، لأن ذلك يعتمد على سرعة المعالج وعوامل خارجية أخرى. بدلاً من ذلك، نقوم بعدّ عدد العمليات الأساسية التي تنفذها الخوارزمية.

2. **تعقيد المساحة :(Space Complexity)** وهو مقياس لمقدار الذاكرة (Memory)التي تحتاجها الخوارزمية لتكتمل كدالة في حجم المدخلات.

في معظم الحالات، يكون **تعقيد الوقت** هو الشاغل الأكبر، خاصة مع توفر الذاكرة بسعات كبيرة في الأجهزة الحديثة.

كيفية حساب تعقيد الوقت (مثال بسيط)

لنفترض أن لدينا مهمة بسيطة: إيجاد العنصر الأكبر في مصفوفة (قائمة) غير مرتبة تحتوي على nمن العناصر.

- **الخوارزمية:**

1. نفترض أن العنصر الأول هو الأكبر.
2. نمر على بقية عناصر المصفوفة (من العنصر الثاني حتى الأخير).
3. في كل مرة، نقارن العنصر الحالي بالعنصر الذي افترضنا أنه الأكبر.
4. إذا كان العنصر الحالي أكبر، نحدث افتراضنا.
5. بعد الانتهاء من المرور على كل العناصر، يكون العنصر الذي احتفظنا به هو الأكبر.

- **التحليل:**

- إذا كان حجم المصفوفة هو n، فسنحتاج إلى إجراء n-1مقارنة في أسوأ الأحوال.

- كلما زاد حجم n، زاد عدد العمليات بشكل مباشر. نقول إن هذه الخوارزمية لها **تعقيد وقت خطي.(Linear Time Complexity)**

حالات التحليل: الأفضل، الأسوأ، والمتوسط

أداء الخوارزمية قد يختلف ليس فقط بناءً على حجم المدخلات، ولكن أيضاً على طبيعة هذه المدخلات نفسها. لذلك، نحلل الأداء عادةً في ثلاث حالات:

1. **الحالة الأفضل :(Best Case)** هي السيناريو الذي يؤدي إلى أقل عدد من العمليات. على سبيل المثال، في خوارزمية البحث عن عنصر في قائمة، تكون الحالة الأفضل هي العثور على العنصر من المحاولة الأولى.
2. **الحالة الأسوأ :(Worst Case)** هي السيناريو الذي يؤدي إلى أقصى عدد من العمليات. في مثال البحث، تكون الحالة الأسوأ هي عدم وجود العنصر في القائمة، مما يضطرنا للبحث حتى النهاية .**هذه هي الحالة الأكثر أهمية في التحليل،** لأنها تعطينا ضماناً للحد الأعلى لأداء الخوارزمية.
3. **الحالة المتوسطة :(Average Case)** تمثل الأداء المتوقع للخوارزمية في المتوسط. حسابها قد يكون معقداً رياضياً، لكنها مفيدة لفهم الأداء النموذجي.

خلاصة :تحليل التعقيد هو أداة أساسية لتقييم الخوارزميات. من خلال فهم تعقيد الوقت والمساحة، وحالات الأداء المختلفة، يمكننا اتخاذ قرارات واضحة لتصميم حلول برمجية فعالة وقابلة للتطوير.

لترسيخ فهمنا لكيفية تحليل الخوارزميات، دعونا نتناول بعض الأمثلة العملية التي توضح مفاهيم تعقيد الوقت وحالات التحليل المختلفة.

مثال توضيحي (1): تحليل خوارزمية البحث في مصفوفة مرتبة

لنفترض أن لدينا مصفوفة (قائمة) **مرتبة** من الأرقام، ونريد البحث عن رقم معين فيها. أحد أشهر الخوارزميات المستخدمة لهذا الغرض هو **البحث الثنائي (Binary Search)**. فكرة هذه الخوارزمية هي:

1. تبدأ بالنظر إلى العنصر الموجود في منتصف المصفوفة.
2. إذا كان هذا العنصر هو الرقم الذي نبحث عنه، نكون قد انتهينا.
3. إذا كان الرقم الذي نبحث عنه **أصغر** من عنصر المنتصف، فإننا نتجاهل النصف الأيمن (الأكبر) من المصفوفة ونكرر البحث في النصف الأيسر فقط.
4. إذا كان الرقم الذي نبحث عنه **أكبر** من عنصر المنتصف، فإننا نتجاهل النصف الأيسر (الأصغر) ونكرر البحث في النصف الأيمن فقط.
5. نستمر في تقسيم المصفوفة إلى نصفين في كل خطوة حتى نجد العنصر أو تنتهي عناصر المصفوفة.

الآن، لنحلل حالات الأداء لهذه الخوارزمية:

- **الحالة الأفضل:(Best Case)**
  - **السيناريو :**أن يكون الرقم الذي نبحث عنه هو بالضبط العنصر الموجود في منتصف المصفوفة.
  - **التحليل :**في هذه الحالة، سنجد العنصر من **المقارنة الأولى** .هذا هو أسرع سيناريو ممكن، ويتطلب عملية واحدة فقط بغض النظر عن حجم المصفوفة.

- **الحالة الأسوأ:(Worst Case)**

- **السيناريو :**أن يكون الرقم الذي نبحث عنه غير موجود في المصفوفة على الإطلاق، أو أن يكون في آخر مكان ممكن نصل إليه بعد تكرار عمليات التقسيم.

- **التحليل :**في كل خطوة، نحن نقلل حجم البحث إلى النصف. سنستمر في تقسيم المصفوفة مراراً وتكراراً حتى لا يتبقى لدينا سوى عنصر واحد للمقارنة. عدد المرات التي يمكننا فيها تقسيم nمن العناصر إلى النصف يُمثَّل رياضياً بـ **لوغاريتم الأساس 2 للعدد) n** ويرمز له بـ .($\log_2(n)$)هذا هو أقصى عدد من العمليات التي سنحتاجها.

- **الحالة المتوسطة:(Average Case)**

- **السيناريو :**أن يكون العنصر موجوداً في مكان ما عشوائياً داخل المصفوفة.

- **التحليل :**رياضياً، يتطلب التحليل حساب متوسط عدد المقارنات لكل الاحتمالات الممكنة. النتيجة ستكون قريبة جداً من الحالة الأسوأ، أي أنها أيضاً لوغاريتمية .($\log_2(n)$) الفكرة هي أن الخوارزمية في المتوسط تتخلص من نصف البيانات في كل خطوة، مما يجعلها سريعة جداً.

مثال توضيحي (2): تحليل كود جمع عناصر مصفوفة

لنفترض أننا نريد كتابة برنامج بسيط بلغة البايثون يقوم بجمع كل الأرقام في مصفوفة حجمها .n

```
def sum_array(numbers):
 total = 0  # 1. عملية تهيئة (تُنفذ مرة واحدة)
 for number in numbers: # 2. حلقة تكرارية تمر على كل عنصر
   total += number    # 3. عملية جمع (تُنفذ n مرة)
 return total
```

#مثال للاستخدام

```
my_list = [1, 2, 3, 4, 5] # هنا n = 5
result = sum_array(my_list)
print(f"The sum is: {result}")
```

- **تحليل تعقيد الوقت:**
- **لماذا هذا هو التعقيد؟** لتحليل الكود، نقوم بعدّ العمليات الأساسية التي تعتمد على حجم المدخلات (nحجم قائمة numbers).

1. عملية total = 0 تُنفذ مرة واحدة فقط، بغض النظر عن حجم n.
2. الحلقة التكرارية for number in numbers ستدور nمن المرات، مرة لكل عنصر في القائمة.
3. داخل الحلقة، عملية الجمع total += number ستُنفذ أيضاً nمن المرات.

- إذن، عدد العمليات يتناسب طردياً مع .nإذا كان لدينا 10 عناصر، ستدور الحلقة 10 مرات. إذا كان لدينا مليون عنصر، ستدور مليون مرة.
- لهذا السبب، نقول إن **تعقيد الوقت لهذا الكود هو تعقيد خطي (Linear Time Complexity)**.

لماذا نركز عادةً على "الحالة الأسوأ" عند تحليل الخوارزميات؟

قد يبدو من المنطقي الاهتمام بالحالة المتوسطة لأنها تمثل الأداء اليومي، لكن التركيز على الحالة الأسوأ هو المعيار الذهبي في تحليل الخوارزميات لعدة أسباب حاسمة:

1. **ضمان الأداء :(Performance Guarantee)** الحالة الأسوأ تخبرنا بالحد الأعلى لأداء الخوارزمية. إنها بمثابة "كفالة" بأن أداء الخوارزمية **لن يكون أسوأ من هذا الحد أبداً**، بغض النظر عن طبيعة المدخلات. هذا مهم جداً في التطبيقات الحساسة للوقت (مثل أنظمة التحكم في الطائرات أو المعاملات المالية) حيث يجب ضمان الاستجابة خلال فترة زمنية محددة.
2. **سهولة التحليل :(Simplicity of Analysis)** في كثير من الأحيان، يكون تحليل الحالة الأسوأ أسهل رياضياً من تحليل الحالة المتوسطة، التي قد تتطلب معرفة بالتوزيع الاحتمالي للمدخلات وحسابات معقدة.
3. **الأداء المتقارب :(Close Performance)** بالنسبة للعديد من الخوارزميات المهمة (مثل البحث الثنائي والفرز السريع)، تكون الحالة المتوسطة قريبة جداً من الحالة الأسوأ، لذا فإن تحليل الحالة الأسوأ يعطينا فكرة جيدة عن الأداء العام أيضاً.